

Signal Processing Implementation in a Laser-Based Scanning Object Detection System

Abstract

This article presents the development of a laser-based scanning object detection system integrated with a MATLAB application for real-time visualization and signal processing. The prototype combines an ESP32 microcontroller, a laser time-of-flight distance sensor, and an SG90 micro servo motor to perform angular scanning and distance acquisition. The collected data is transmitted to MATLAB, where it is visualized in spatial, time-domain, and frequency-domain forms. The most important part of the implementation is the signal processing pipeline applied to the measured distance signal. This pipeline includes moving average filtering, DC offset removal, estimation of sampling frequency from recorded timestamps, and Fast Fourier Transform (FFT) analysis. The developed system demonstrates how a compact embedded sensing platform can be combined with software-based processing to improve measurement interpretation and provide a foundation for future extensions such as velocity estimation, acceleration analysis, and intelligent object tracking.

1. Introduction

Object detection and distance measurement are essential in robotics, automation, and smart sensing systems. In many practical applications, it is not enough only to detect the presence of an object; it is also necessary to determine its position, observe how the measured distance changes with time, and identify whether the measured behavior contains meaningful patterns. For this reason, measurement hardware is often combined with software-based processing and visualization.

The project described in this article focuses on a radar-like scanning detection system based on a laser distance sensor. Although the developed prototype is not a classical radio-frequency radar, it performs spatial scanning in a similar way by combining angular position with distance measurements to build a map of the observed environment. The acquired data is sent to MATLAB for real-time visualization and signal processing.

The main engineering contribution of the project lies in the signal processing stage. Real measured data is affected by fluctuations caused by sensor noise, timing irregularities, and small disturbances related to mechanical scanning. To obtain a more useful interpretation of the measurements, the software applies filtering and frequency-domain analysis to the distance signal. These processing steps transform the raw measurements into a form that is clearer and more suitable for engineering analysis.

2. Project Objective

The objective of the project was to design and implement a compact scanning object detection system capable of measuring distance with a laser sensor, sweeping the environment through angular motion, and transmitting the acquired data to a MATLAB application for analysis.

The system was intended to provide three main types of output. First, it should display a radar-like spatial representation of detected points. Second, it should show the measured distance signal in the time domain, including both raw and filtered versions. Third, it should analyze the frequency content of the distance signal using FFT so that periodic behavior can be identified.

A secondary objective was to create a software and hardware structure that could later be extended toward velocity and acceleration estimation. In the current version, the focus is on reliable acquisition, visualization, and signal processing of the measured distance data.

3. System Architecture

The developed system consists of three layers: sensing hardware, embedded control, and software-based analysis. The sensing layer measures the distance to an object using a laser time-of-flight sensor. The scanning layer is implemented with a servo motor that rotates the sensor through a selected angular range. As the angle changes, new measurements are collected from different directions, producing a radar-like scanning effect.

The embedded control layer is built around an ESP32 development board. The ESP32 communicates with the sensor, drives the servo motor, and sends the measured angle and distance values to the computer through serial communication. This role makes the microcontroller the central coordinator of the measurement process.

The software layer is implemented in MATLAB App Designer. The application receives incoming data, extracts numerical values, assigns timestamps, stores recent measurement history, and continuously updates multiple plots. This division of tasks makes the overall system easy to understand and extend.

4. Hardware Components and Their Roles

The central hardware component of the system is the ESP32 microcontroller board. It controls the scanning sequence, manages communication with the sensor, drives the servo motor, and transmits measurement data to the computer. The ESP32 was selected because it is well suited for embedded prototyping and real-time communication tasks.

Distance measurement is performed using a compact laser time-of-flight sensor module of the VL53L1X type or a similar I2C-based ranging module. This sensor measures distance without contact by emitting light and evaluating the reflected signal. It serves as the primary source of the distance data later processed in MATLAB.

Angular scanning is implemented using an SG90 9 g micro servo motor. The servo rotates the sensor through the measurement field, allowing a single-direction distance sensor to behave as a scanning system. By combining the distance value with the servo angle, the system can estimate the spatial location of detected objects.

A solderless breadboard and jumper wires were used for rapid prototyping and testing. USB connection to the ESP32 provides both system power and serial communication with MATLAB. Although this construction method is appropriate for development and demonstration, a future version could use a dedicated PCB and a more stable mechanical sensor mount.

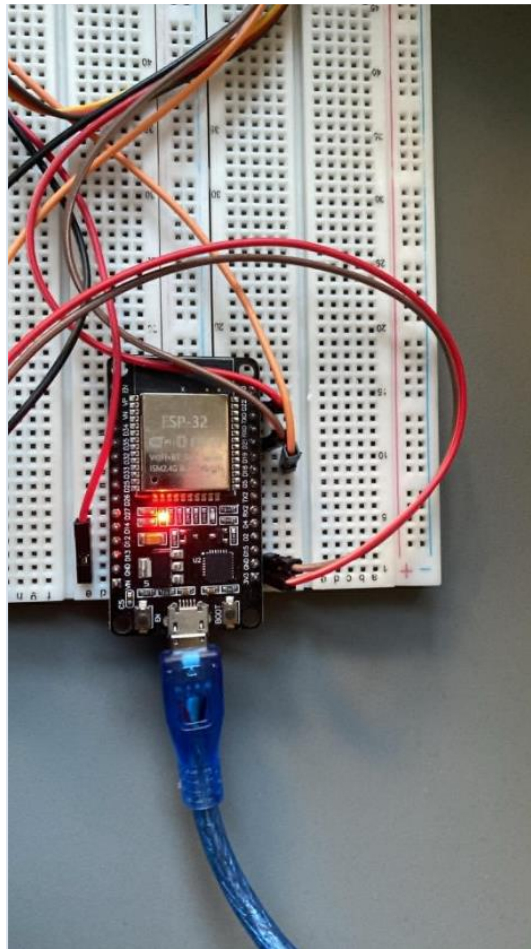


Figure 1. ESP32 development board used as the central embedded controller.

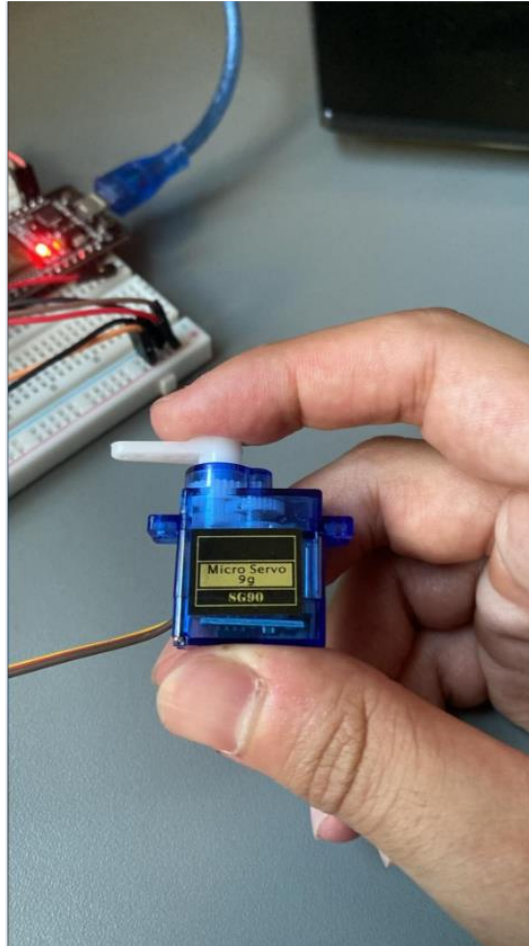


Figure 2. SG90 micro servo motor used to provide angular scanning.

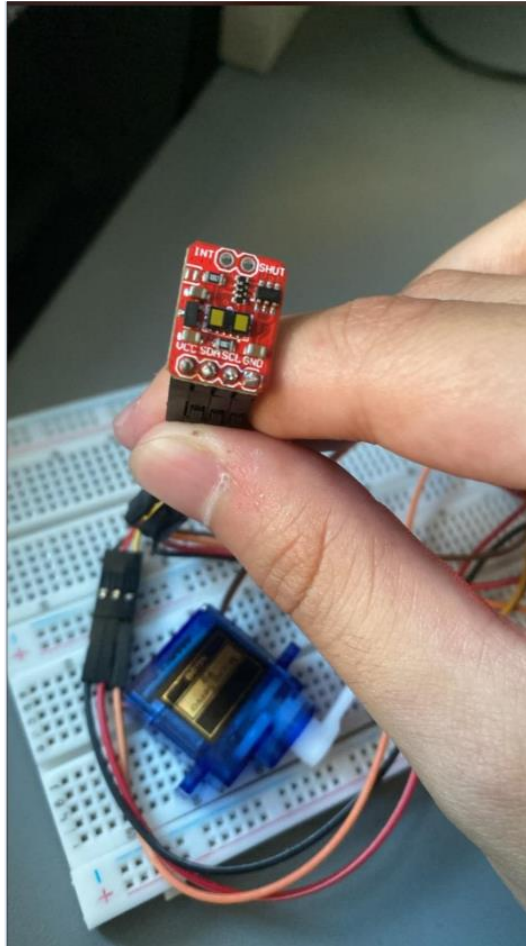


Figure 3. Laser time-of-flight sensor module used for distance measurement.

5. Data Acquisition Method

The MATLAB application begins acquisition when the user presses the Start button. At that moment, MATLAB opens a serial connection to the ESP32, clears previous buffers, sends a start command, and enters a loop in which it continuously reads incoming lines from the embedded system.

Each received line contains the current scan angle and the corresponding measured distance. After reception, the application extracts the numerical values and stores them in arrays. In addition to angle and distance, the software records elapsed time relative to the beginning of the measurement. This timestamp is essential for both time-domain and frequency-domain analysis.

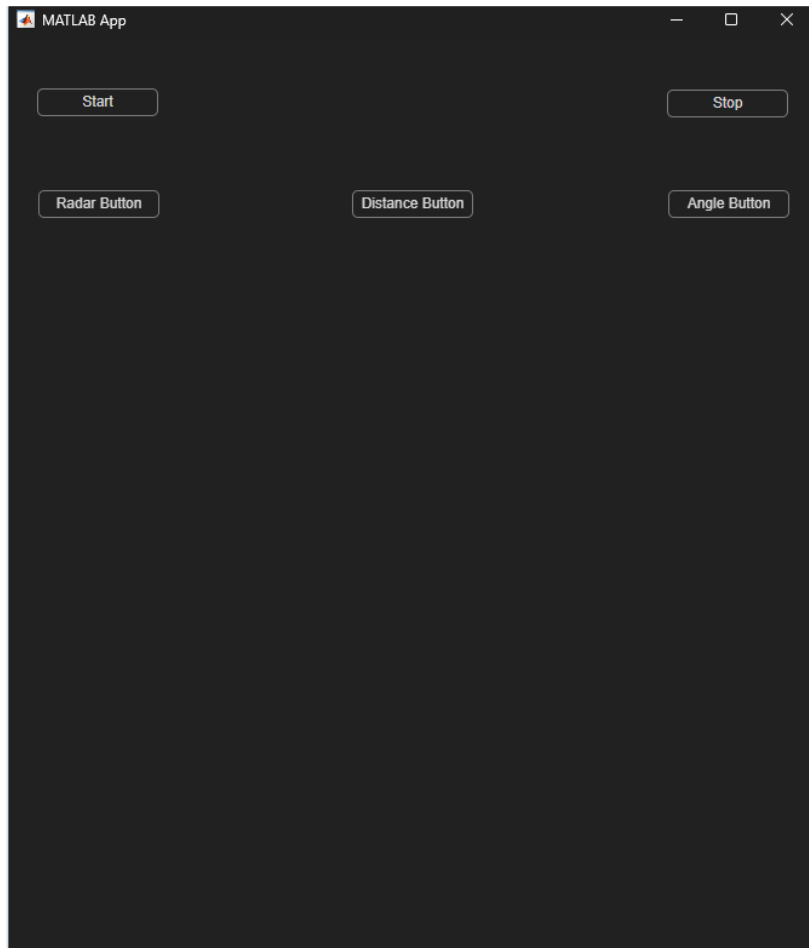


Figure 4. Initial MATLAB App interface showing the main control buttons and display modes.

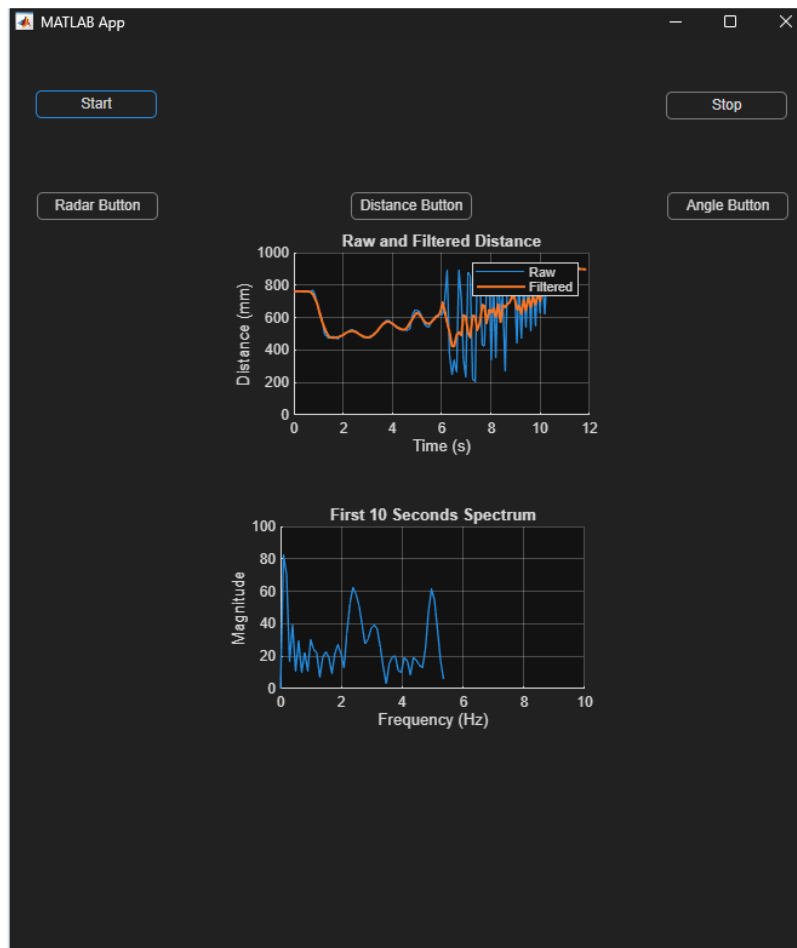


Figure 5. Distance mode showing the raw and filtered distance signals together with the first 10 seconds spectrum.

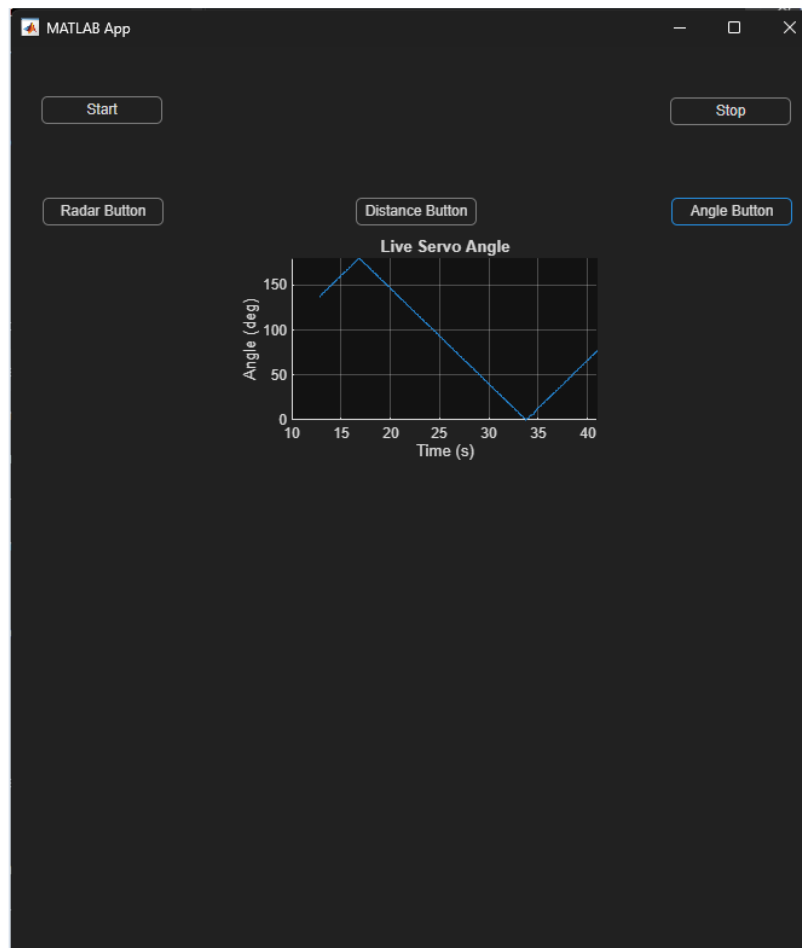


Figure 6. Angle mode showing the live servo angle as a function of time.

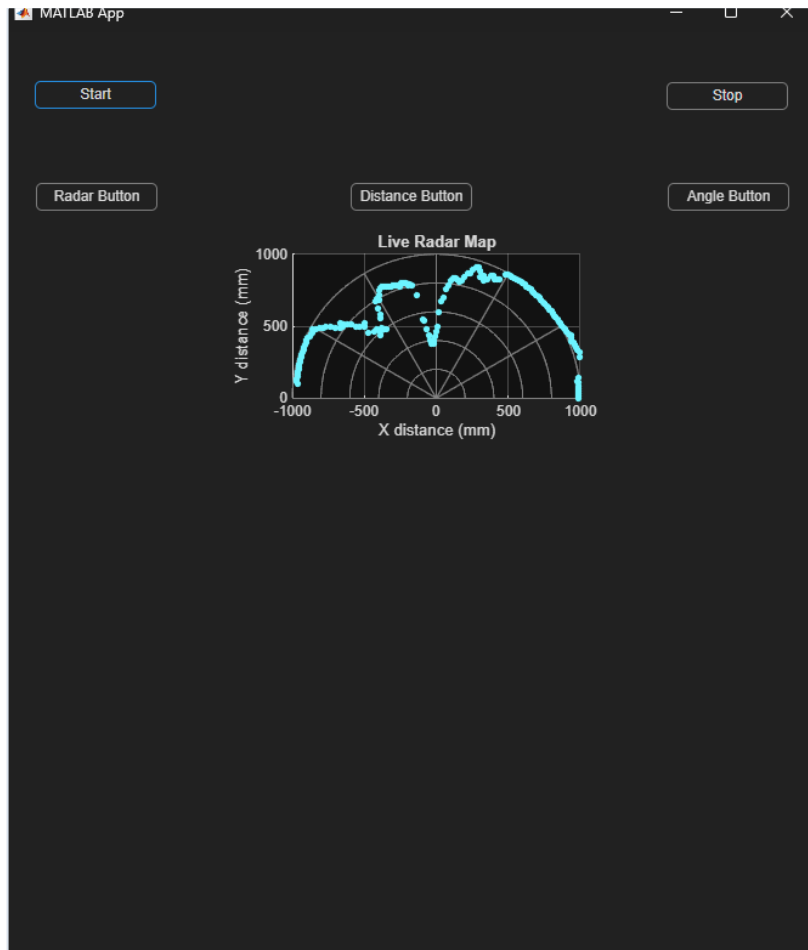


Figure 7. Radar mode showing the live radar-like map generated from angle and distance data.

Internally, the angle is stored in radians because trigonometric functions are required for radar-map visualization. The measured distance is stored in millimeters. The timestamp array is used both for plotting the distance signal and for estimating the effective sampling frequency during FFT analysis.

6. MATLAB Application and Visualization Interface

A dedicated MATLAB application was developed to provide real-time control, visualization, and signal processing. The interface includes Start and Stop buttons together with three main display modes: Radar, Distance, and Angle. This structure allows the user to begin or terminate acquisition and then switch between different measurement views depending on the analysis goal.

In Radar mode, the system displays a live radar-like map of detected points. The measured polar coordinates are transformed into Cartesian coordinates and plotted on a semicircular grid. This mode is useful for observing where objects appear within the scanning field.

In Distance mode, the upper plot displays both the raw and the filtered distance signals as a function of time. The lower plot displays the spectrum of the first ten seconds of the acquired distance data. This makes Distance mode the most important interface view for signal processing.

In Angle mode, the current servo angle is plotted against time. This graph makes it possible to verify the scanning motion and confirm that the radar-like map is associated with the expected angular sweep of the servo motor.

7. Spatial Representation of Measured Data

The radar-like map is generated by transforming measured polar coordinates into Cartesian coordinates. If the measured distance is denoted by r and the scan angle by θ , the position is computed using $x = r \cos(\theta)$ and $y = r \sin(\theta)$.

This transformation allows each measurement to be displayed as a point in a two-dimensional plane. As the servo sweeps across the measurement range and new samples are collected, the set of points gradually forms a visual map of the observed environment.

From an engineering point of view, this stage is important because it combines the two measured variables distance and angle into a single spatial representation. The result is intuitive for the user and useful for checking whether the scanning process is functioning correctly.

8. Signal Processing Implementation

The most important processing stage in the project is the analysis of the distance signal as a function of time. The raw signal obtained from the sensor contains useful information about object position, but it also includes fluctuations caused by measurement noise, mechanical motion, and timing irregularities. For this reason, software-based signal processing is applied before detailed interpretation.

8.1 Raw Distance Signal

The raw distance signal is plotted directly against time. This representation shows how the measured distance changes during system operation. However, the graph may contain rapid sample-to-sample changes that do not necessarily correspond to real object motion. These fluctuations make the signal more difficult to interpret visually and motivate the use of filtering.

8.2 Moving Average Filtering

To reduce short-term fluctuations, a moving average filter is applied to the distance data. In the MATLAB application, the filter is implemented with a five-sample window. Each filtered sample is obtained from the average of neighboring values, which produces a smoother signal.

This filter behaves as a simple low-pass filter. It suppresses high-frequency variations while preserving slower changes that are more likely to represent the actual trend of the measured object distance. The filtered signal remains a distance signal; it is not a new physical quantity, but rather a smoothed estimate of the original measurement.

The choice of a five-sample window represents a compromise between smoothing and responsiveness. A larger window would reduce fluctuations more strongly, but it would also make the signal less responsive to real changes. A smaller window would preserve fast variations but would be less effective in reducing noise.

8.3 Selection of the Signal Segment for Spectral Analysis

For frequency-domain analysis, the application selects the distance data recorded during the first ten seconds of acquisition. This fixed segment provides a consistent basis for FFT computation and avoids the complication of continually extending the spectrum as more data arrives.

The number of samples used in FFT is not fixed manually. Instead, it is determined by how many valid measurements are collected during those first ten seconds. In the software, this number is denoted by N and equals the length of the selected signal segment.

8.4 DC Offset Removal

Before FFT is applied, the mean value of the selected distance signal is subtracted. This operation removes the DC offset. In practical terms, the DC offset corresponds to the average distance level of the object from the sensor.

If the mean were not removed, the resulting spectrum would contain a large peak at zero frequency. That peak would mainly reflect the average object position rather than the actual changing part of the signal. By centering the signal around zero, the FFT analysis becomes focused on the variation in distance, which is the quantity of real interest for motion interpretation.

8.5 Estimation of Sampling Frequency

An important feature of the implementation is that the sampling frequency is not assumed to be constant. Because the data is acquired through real-time serial communication, small variations in sampling interval are expected. The application therefore estimates the effective sampling frequency directly from the recorded timestamps.

This is done by calculating the time difference between consecutive samples, taking the average of those differences, and then using the inverse of the average interval as the estimated sampling frequency. This step is essential because the FFT frequency axis must be expressed in real units of hertz rather than arbitrary sample indices.

8.6 Fast Fourier Transform

After the distance signal segment has been selected, the mean removed, and the sampling frequency estimated, the Fast Fourier Transform is calculated. The FFT converts the signal from the time domain into the frequency domain. In the time domain, the signal is described as distance changing over time. In the frequency domain, it is described by the frequencies that contribute to that change.

The FFT output is complex because each frequency component contains both magnitude and phase information. Magnitude describes how strong a frequency component is, whereas phase describes its relative shift. For visualization in this project, only the magnitude spectrum is used. This is obtained by taking the absolute value of the FFT output and normalizing it.

Because the original distance signal is real-valued, the FFT spectrum is symmetric. For this reason, a one-sided spectrum containing the positive-frequency components is displayed. This form is simpler and more suitable for practical engineering interpretation.

8.7 Interpretation of the Spectrum

The resulting spectrum reveals the dominant frequencies present in the distance variation. If the measured object motion is periodic, it appears as a peak in the magnitude spectrum. The corresponding period can then be estimated from the reciprocal relationship $T = 1/f$.

In the current implementation, the FFT does not directly measure velocity or acceleration. Instead, it identifies periodic components in the distance signal. Velocity and acceleration would require derivative-based calculations applied to the time-domain measurements. Nevertheless, the FFT stage provides valuable insight into the dynamic behavior of the measured signal.

9. Discussion

The developed prototype demonstrates how a relatively simple combination of hardware and software can become a meaningful sensing and analysis platform. The system does not only measure distance; it also transforms those measurements into spatial and signal-based representations that are more informative to the user.

The moving average filter improves signal readability and behaves as a low-pass smoothing stage. The DC offset removal step ensures that the frequency spectrum reflects true signal variation rather than average object position. The timestamp-based estimation of sampling frequency makes the FFT implementation more realistic, since it does not rely on an ideal fixed sample rate.

At the same time, the current system has limitations. The sampling process is not perfectly uniform, the moving average filter is relatively simple, and the present implementation focuses on distance analysis rather than direct motion estimation. These limitations, however, also define clear directions for future work.

10. Future Work

A natural next step for the project is the estimation of velocity and acceleration from the measured distance signal. This could be achieved by differentiating the filtered distance with respect to time, potentially after additional smoothing or more advanced digital filtering.

Other improvements include designing a filter with an explicitly chosen cutoff frequency, improving the uniformity of sampling, using a more robust mechanical mounting arrangement, and extending the MATLAB interface with data logging and automated analysis features.

Further developments may also include threshold-based object tracking, adaptive filtering, and more advanced spectral techniques such as windowing or overlapping analysis segments.

11. Conclusion

This project presents a laser-based scanning object detection system integrated with MATLAB-based visualization and signal processing. The hardware combines an ESP32 controller, a time-of-flight distance sensor, and an SG90 servo motor. The software environment receives angle and distance data, maps object position in a radar-like display, smooths the distance signal in the time domain, and analyzes its periodic content in the frequency domain.

The most significant contribution of the work is the signal processing implementation. A moving average filter is used to stabilize the measured distance signal, DC offset removal is applied before FFT, the sampling frequency is estimated from real timestamps, and the resulting one-sided magnitude spectrum is used to reveal dominant periodic behavior.

The current prototype provides a solid base for future extensions involving velocity estimation, acceleration analysis, and more advanced object tracking. As a result, the project serves both as a practical embedded measurement system and as an educational example of how signal processing improves interpretation of real-world sensor data.